Square Integration

1. Overview

1.1 Description

Square Integration connects **Square** (appointment scheduling and POS platform) with **Newo Platform**.

It enables:

- Checking real-time appointment availability from Square Bookings
- Booking appointments with automatic customer creation in Square
- Cancelling existing appointments
- Synchronizing service catalogs (services, variations, pricing) from Square
- LLM-powered service matching from natural language requests
- Team member assignment based on availability per slot
- OAuth 2.0 authentication with automatic token refresh

This integration is designed for **service businesses** (salons, spas, healthcare, etc.) who need **automated appointment scheduling through a conversational AI agent** (voice or chat).

1.2 Use Cases

- When a client asks about availability → Query Square Bookings for open slots with matched service
- When a client requests a specific service → LLM matches natural language to service catalog variations
- When a client wants to book → Find or create customer, assign team member, create booking
- When a client wants to cancel → Cancel booking via Square API with idempotency

1.3 Supported Features

Feature	Supported	Notes
Check Availability	✅	By date, time, and service type via Square Bookings API
Book Appointment	✅	With auto customer creation and team member assignment
Cancel Appointment	✅	By booking ID with idempotency key
Service Catalog Sync	✅	Auto-sync on session start, filters APPOINTMENTS_SERVICE
LLM Service Matching	✅	Gemini matches natural language → service variation ID
Team Member Assignment	✅	Auto-selects available team member per slot
Multi-Location Support	✅	Location selection via enum attribute
Idempotent Operations	✅	UUID keys for booking, customer creation, cancellation
Reschedule	❌	Cancel + rebook as workaround
Payment Processing	❌	Not implemented

2. Requirements

Before installation:

- Active **Square** account with Bookings enabled
- **Square OAuth App** with Client ID and Secret
- At least one **location** configured in Square
- Services defined in Square **Catalog** (product type: APPOINTMENTS_SERVICE)

3. How to Setup

3.1 Step 1 — Authorize in Square

1. In Newo platform, set `square_client_id` and `square_secret_id`
2. Click the Square authorization link provided by the platform
3. Log in to your Square account and select the sub-account
4. Authorize the Newo app (requested scopes include appointments, customers, items, employees, merchant profile)
5. Copy the **authorization code** from the redirect URL
6. Paste the code into the `square_oauth_code` attribute

3.2 Step 2 — Connect in Platform

1. Open **Newo → Projects**
2. Set the following attributes in **Builder / Attributes**:

Attribute	Required	Description
`square_client_id`	✅	OAuth2 Application ID
`square_secret_id`	✅	OAuth2 Application Secret
`square_oauth_code`	✅	One-time OAuth authorization code
`square_locations`	✅	Selected location (auto-populated as enum after auth)
`square_base_url`	❌	API base URL (default: `https://connect.squareup.com`)
`square_login_url`	❌	Login URL (default: `https://app.squareup.com`)
`square_enable_slot_check`	❌	Enable availability checking (default: `True`)
`square_enable_booking`	❌	Enable booking capability (default: `True`)
`square_enable_cancellation`	❌	Enable cancellation capability (default: `True`)

3. Click **Save + Publish All**
4. On publish, the SetupFlow automatically:
    - Exchanges the OAuth code for access and refresh tokens
    - Retrieves merchant locations and populates the location enum
    - Creates HTTP connectors for Square API
    - Injects booking and availability payload schemas for the agent
5. On first conversation, the SetupCatalogFlow syncs the service catalog

4. How to Use

4.1 How the Integration Works

The integration uses Square **Bookings API** for scheduling, **Catalog API** for services, and **Customers API** for contact management. Availability is computed via the bookings availability search, and services are matched from the synced catalog using LLM.

- A **Trigger** is a system event that starts a flow
- An **Action** is the API operation performed in Square

Available Triggers

Trigger	Event	Description
Check Availability	`get_available_slots`	When the agent needs to look up open slots
Book Appointment	`book_slot`	When the agent confirms a booking
Cancel Appointment	`convo_agent_cancel_booking`	When the agent processes a cancellation
Catalog Sync	`session_started`	Auto-sync service catalog on conversation start
Setup	`project_publish_finish`	Initializes integration on deploy

Available Actions

- **Search Availability** — Find open slots by service and date range (POST `/v2/bookings/availability/search`)
- **Search Customer** — Find customer by phone (POST `/v2/customers/search`)
- **Create Customer** — New customer record (POST `/v2/customers`)
- **Create Booking** — Schedule appointment with team member (POST `/v2/bookings`)
- **Cancel Booking** — Cancel appointment (POST `/v2/bookings/{id}/cancel`)
- **Search Catalog** — Sync service variations (POST `/v2/catalog/search`)
- **Get Locations** — List merchant locations (GET `/v2/locations`)

4.2 Architecture Diagrams

Overall Sequence

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as Square Integration
    participant T as Square OAuth
    participant API as Square API

    Note over I,T: Setup (on publish)
    A->>I: project_publish_finish
    I->>T: POST /oauth2/token (code)
    T-->>I: access_token + refresh_token
    I->>API: GET /v2/locations
    API-->>I: Location list (populate enum)
```

```
    Note over I,API: Catalog Sync (on session start)
    A->>I: session_started
    I->>API: POST /v2/catalog/search (ITEM, APPOINTMENTS_SERVICE)
    API-->>I: Service variations
    I->>I: Store in square_catalog

    Note over U,API: Availability Check
    U->>A: "Any openings for a manicure tomorrow?"
    A->>I: get_available_slots (date, time, service_request)
    I->>I: LLM match "manicure" -> service_variation_id
    I->>API: POST /v2/bookings/availability/search (location, service, time range)
    API-->>I: Available slots + team members
    I->>I: Store team member map per slot
    I->>A: result_success (availability[])
    A->>U: "Here are the available times..."

    Note over U,API: Booking
    U->>A: "Book 2:00 PM please"
    A->>I: book_slot (bookingDateTime)
    I->>API: POST /v2/customers/search (phone)
    alt Customer not found
        I->>API: POST /v2/customers (name, phone)
        API-->>I: customer_id
    end
    I->>I: Look up team_member_id for selected time
    I->>API: POST /v2/bookings (location, customer, service, team_member, start_at)
    API-->>I: booking_id
    I->>A: result_success (booking_id)
    A->>U: "Your appointment is confirmed!"

    Note over U,API: Cancellation
    U->>A: "Cancel my appointment"
    A->>I: convo_agent_cancel_booking (booking_id)
    I->>API: POST /v2/bookings/(id)/cancel (idempotency_key)
    API-->>I: Confirmation
    I->>A: result_success
    A->>U: "Your appointment has been cancelled"
```

AvailabilityFlow

```
flowchart TD
    E["get_available_slots"] --> GATE{{"Slot check<br>enabled?"}}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| LLM["LLM match:<br>service_request →<br>service_variation_id<br>
(temp=0.2)"]
    LLM --> UTC["Convert date/time<br>to UTC range<br>(+ 5 days window)"]
    UTC --> API["POST /v2/bookings<br>/availability/search<br>{location, service,
<br>time range}"]
    API --> OK{{"Success?"}}
    OK -->|"No"| ERR["ResultError"]
    OK -->|"Yes"| PARSE["Parse slots:<br>extract team_member_id<br>per time slot"]
```

```
    PARSE --> TZ["Convert to local<br>timezone (AM/PM)"]
    TZ --> STORE["Store team member<br>map in persona:<br>square_free_teamembers"]
    STORE --> OUT["result_success<br>{availability[]}"]
```

BookingFlow

```
flowchart TD
    E["book_slot"] --> GATE{{"Booking<br>enabled?"}}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| PHONE{{"Phone<br>available?"}}
    PHONE -->|"No"| ASK["urgent_message:<br>ask for phone"]
    PHONE -->|"Yes"| SEARCH["POST /v2/customers<br>/search {phone}"]
    SEARCH --> FOUND{{"Customer<br>found?"}}
    FOUND -->|"No"| CREATE["POST /v2/customers<br>{name, phone}<br>(idempotency
key)"]
    CREATE --> CID["customer_id"]
    FOUND -->|"Yes"| CID
    CID --> TEAM["Look up team_member_id<br>from square_free_teamembers<br>for
selected time"]
    TEAM --> UTC["Convert time to UTC"]
    UTC --> BOOK["POST /v2/bookings<br>{location, customer,<br>service_variation,
<br>team_member, start_at}<br>(idempotency key)"]
    BOOK --> OUT["result_success<br>{booking_id}"]
```

CancellationFlow

```
flowchart TD
    E["convo_agent_cancel_booking"] --> GATE{{"Cancellation<br>enabled?"}}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| ID["Extract booking_id<br>from payload.meta"]
    ID --> API["POST /v2/bookings<br>/{booking_id}/cancel<br>{idempotency_key}"]
    API --> OK{{"Success?"}}
    OK -->|"No"| ERR["ResultError"]
    OK -->|"Yes"| OUT["result_success"]
```

SetupCatalogFlow

```
flowchart TD
    E["session_started"] --> TOKEN{{"Access token<br>exists?"}}
    TOKEN -->|"No"| SKIP["Skip"]
    TOKEN -->|"Yes"| API["POST /v2/catalog/search<br>{ITEM type,<br>not deleted}"]
    API --> FILTER["Filter items:<br>product_type ==<br>APPOINTMENTS_SERVICE"]
    FILTER --> LOC["Check location:<br>present_at_all_locations<br>OR in
square_locations"]
    LOC --> EXTRACT["Extract variations:<br>id, name, description,<br>service_name"]
    EXTRACT --> STORE["Store as<br>square_catalog JSON"]
```

4.3 Where to Test

Testing can be done through the **Newo conversation interface** (chat or voice). Each flow has dedicated webhook test endpoints for connectivity validation.

4.4 Testing and Verification

To test the integration:

1. **Setup**: Publish the project and verify OAuth token exchange + location discovery
2. **Catalog**: Start a conversation — verify `square_catalog` is populated with service variations
3. **Availability**: Ask "Any openings for [service] tomorrow?" — verify slots returned with team members
4. **Booking**: Complete a booking — verify appointment created in Square with correct service and team member
5. **Cancellation**: Cancel a booking — verify appointment deleted from Square

If no action occurs:

- Ensure OAuth credentials are set (`square_client_id` , `square_secret_id`)
- Verify `square_access_token` was obtained after setup
- Confirm `square_locations` is selected (should be auto-populated as enum)
- Check `square_catalog` is not empty (services must exist in Square as APPOINTMENTS_SERVICE)
- Verify feature flags are enabled (`square_enable_slot_check` , `square_enable_booking` , `square_enable_cancellation`)
- Re-publish the project if credentials were changed

> *Note: This integration creates real appointments in Square. Bookings and cancellations are reflected in the live Square system.*

5. FAQ

Q: How does the agent match services from natural language? A: The integration uses Gemini 2.5 Flash (temperature=0.2) to match the user's service request (e.g., "manicure") against the synced `square_catalog` of service variations. It returns the matching `service_variation_id` used for the availability search and booking.

Q: How are team members assigned to appointments? A: During the availability search, Square returns which team members are available for each slot. The integration stores this mapping (`square_free_teamembers`) in the persona. When booking, the correct team member is looked up for the selected time.

Q: Why are idempotency keys used? A: All write operations (customer creation, booking, cancellation) use UUID idempotency keys to prevent duplicate operations if requests are retried. This is a Square API best practice.

Q: When is the service catalog synced? A: The catalog syncs automatically on `session_started` (every new conversation). It filters for items with `product_type: APPOINTMENTS_SERVICE` that are available at the selected location.

Q: Can I connect multiple Square locations? A: The integration supports one location per instance. During setup, available locations are fetched and presented as an enum (`square_locations`) in the format `"id | name | timezone"` . Select the desired location after initial auth.

Q: What happens if the OAuth token expires? A: The integration detects HTTP 401 responses and automatically refreshes the token via `/oauth/v2/refresh` using the stored refresh token. Failed requests are retried up to 3 times.

Q: What customer information is needed for booking? A: The customer's phone number is required for lookup/creation. Name (first/last) is used when creating a new customer. The integration splits `full_name` into first and last names automatically.